CodeGraph 从入门到精通

> **版本与核实信息**
>
> - 适用版本：`@colbymchenry/codegraph` v1.6.2（npm 最新稳定版，2026-10-24 核实）
> - 资料来源：GitHub 仓库 colbymchenry/codegraph、官方文档站 colbymchenry.github.io/codegraph（2026-10-24 抓取核实）
> - 支持平台：Windows（x64 / arm64）、macOS（x64 / arm64）、Linux（x64 / arm64）
> - 许可证：MIT
> - 说明：本教程基于上述时间点的官方文档核实编写。CodeGraph 迭代较快，若文中命令与最新文档不一致，以官方文档为准。

目录

- 入门篇
  - 第 1 章　认识 CodeGraph
  - 第 2 章　安装与接入
  - 第 3 章　构建首个代码图谱
- 进阶篇
  - 第 4 章　核心概念
  - 第 5 章　CLI 查询命令详解
  - 第 6 章　配置与定制
- 精通篇
  - 第 7 章　与 AI 智能体深度协作
  - 第 8 章　进阶实战
  - 第 9 章　最佳实践
  - 第 10 章　排错、升级与卸载
- 附录 A　CLI 命令速查表
- 附录 B　术语表

入门篇

第 1 章　认识 CodeGraph

本章解决的问题是：CodeGraph 是什么、它试图解决什么问题、它与传统的代码搜索工具差异在哪里。完成本章学习后，读者能够判断 CodeGraph 是否适合自己的工作流，并理解后续章节涉及的基本术语。

1.1 定位：面向 AI 编程智能体的代码知识图谱

CodeGraph 是一个本地优先（local-first）的代码智能工具。它将整个代码库解析为一张可查询的知识图谱（Knowledge Graph）：图中的节点是符号（symbol，指函数、类、方法、文件等代码实体），边是符号之间的关系（调用、引用、依赖等）。

它的核心使用方式有两种：

1. **作为 MCP（Model Context Protocol）服务器**运行，供 Claude Code、Cursor、Codex CLI、opencode、Gemini CLI、Kiro、GitHub Copilot 等 AI 编程智能体调用；
2. **作为命令行工具（CLI）**，供开发者在终端中直接查询代码结构。

需要强调的一点是：CodeGraph 的全部数据都保存在本机，不需要 API Key，不向任何外部服务发送代码。存储介质是一个本地的 SQLite 数据库，位于项目根目录的 `.codegraph/` 文件夹中。

1.2 它解决的问题

当 AI 编程智能体需要理解一段代码时，如果没有预先建立的结构信息，它只能依靠 `grep`、文件列表和逐个读取文件来拼凑上下文。官方文档引用的一组基准测试（2026-08 复测，Claude Opus 4.8，7 个真实开源仓库，每组取 4 次运行的中位数）显示，使用 CodeGraph 后智能体平均：

- 工具调用次数减少约 88%（最多从 43 次降至 2 次）；
- 完成回答的时间缩短约 53%（最高 3.6 倍）；

- 消耗的 token 减少约 62%；
- 在全部七个测试仓库中，读取文件的次数降为 0。

这些数字说明一个问题：智能体大量的时间与 token 花在「发现代码结构」上，而 CodeGraph 把这一步提前完成并缓存为可查询的图谱。

1.3 核心特性

特性	说明
Rust 内核	解析与提取由编译后的 Rust 引擎完成，覆盖 20 种语言，解析结果与参考引擎逐字节一致；无预编译二进制的平台或存在语法错误的文件会按文件粒度自动回退到参考引擎，图谱结果相同
自动同步	文件监视器使用操作系统原生事件（macOS 的 FSEvents、Linux 的 inotify、Windows 的 ReadDirectoryChangesW），保存文件后通常在数百毫秒内完成增量更新，无需手动重建索引
外科手术式上下文	一次查询返回入口符号、相关符号及逐行源码，附带符号间的调用路径与影响范围（blast radius）
框架感知路由	识别 17 个 Web 框架的路由文件（Django、Flask、FastAPI、Express、NestJS、Laravel、Rails、Spring、ASP.NET 等），将 URL 模式与处理函数在图中直接关联
跨语言桥接	支持 Swift 与 Objective-C 互调、React Native 旧桥与 TurboModules、Fabric 视图组件、Expo Modules 等跨语言调用路径的还原
全文检索	基于 SQLite FTS5 扩展，支持按名称快速查找符号
100% 本地	数据不出本机，无外部依赖服务

1.4 支持的语言

CodeGraph 支持的语言包括（截至 v1.6.2）：TypeScript、JavaScript、ArkTS、Python、Go、Rust、Java、C#、VB.NET、PHP、Ruby、C、C++、CUDA、Objective-C、Metal、Swift、Kotlin、Scala、Dart、Lua、Luau、R、Nix、Erlang、CFML、COBOL、Solidity、Terraform / OpenTofu、Svelte、Vue、Astro、Liquid、Pascal/Delphi。

语言支持由文件扩展名自动识别，无需按语言做任何配置。个别项目若使用了非标准扩展名，可通过 `codegraph.json` 显式映射（见第 6 章）。

1.5 关键术语预览

本教程反复出现的术语在此先给出定义，详细机制见第 4 章。

术语	定义
符号（symbol）	代码中可命名的实体，如函数、类、方法、变量、路由
边（edge）	符号之间的关系，如调用（calls）、引用（references）
索引（index）	由解析与消解生成的图谱数据，存储于项目根目录的 `.codegraph/` 中
MCP（Model Context Protocol）	一个开放协议，允许 AI 智能体以标准方式调用外部工具服务器
消解（resolution）	将符号的使用处与定义处匹配起来的过程，是跨文件调用链得以成立的基础

第 2 章　安装与接入

本章解决的问题是：如何把 CodeGraph 安装到本机，并把它接入正在使用的 AI 编程智能体。完成本章学习后，读者能够在自己的操作系统上完成安装，并让智能体加载 CodeGraph 提供的 MCP 工具。

2.1 安装 CLI

CodeGraph 提供「免 Node.js」的安装脚本（自带运行时）与 npm 两种方式，二者效果相同，任选其一。

方式一：安装脚本（推荐，不依赖 Node.js）

macOS 与 Linux（sh）：

```
curl -fsSL https://raw.githubusercontent.com/colbymchenry/codegraph/main/install.sh | sh
```

Windows（PowerShell）：

```
irm https://raw.githubusercontent.com/colbymchenry/codegraph/main/install.ps1 | iex
```

方式二：npm（需要已有 Node.js 环境）

```
npm install -g @colbymchenry/codegraph
```

安装程序会把 `codegraph` 可执行文件放入 `PATH`，但不会修改当前已打开的终端会话。安装完成后需要打开一个新终端，再执行后续命令。

2.2 接入 AI 编程智能体

CLI 安装完成后，还需要把 CodeGraph 的 MCP 服务器注册到智能体中。这一步由 `codegraph install` 完成：

```
codegraph install
```

该命令会交互式地完成以下工作：

1. 自动检测本机已安装的智能体（Claude Code、Cursor、Codex CLI、opencode、Hermes Agent、Gemini CLI、Antigravity IDE、Kiro、GitHub Copilot），并询问要配置哪些；
2. 询问配置作用于全局（所有项目）还是仅当前项目；
3. 将 MCP 服务器配置写入对应智能体的配置文件；对 Claude Code、Cursor、Codex CLI、opencode、Hermes Agent、Gemini CLI 等，还会在指令文件（`CLAUDE.md` / `AGENTS.md` / `GEMINI.md`）中追加一小段带标记围栏的 CodeGraph 说明，使子智能体与非 MCP 环境也能学会使用 `codegraph explore` 命令；
4. 若目标中包含 Claude Code，自动配置工具免确认权限。

需要特别注意：`codegraph install` 只做「接入」，不做「索引」。它不会扫描任何项目代码；为每个项目建立图谱是第 3 章 `codegraph init` 的职责。一次全局安装对所有项目生效，无需按项目重复运行。

非交互模式（脚本与 CI 场景）

命令	作用
`codegraph install --yes`	自动检测智能体并全局安装，跳过全部确认
`codegraph install --target=cursor,claude --yes`	指定目标智能体列表
`codegraph install --target=auto --location=local`	使用检测到的智能体，配置写入当前项目
`codegraph install --print-config codex`	仅打印某个智能体的配置片段，不写任何文件

安装完成后，重启智能体（或重新打开会话），MCP 服务器即可加载。

2.3 验证安装

```
codegraph version
```

该命令输出版本号。截至本教程核实时，最新版本为 1.6.2。

2.4 升级

```
codegraph upgrade            # 升级到最新版本
codegraph upgrade --check    # 仅检查是否有新版本
```

第 3 章　构建首个代码图谱

本章解决的问题是：如何为一个具体项目建立图谱，并验证它可以使用。完成本章学习后，读者能够独立完成「初始化 → 验证 → 首次查询 → 交给智能体」的完整流程。

3.1 初始化项目

进入目标项目根目录，执行：

```
cd your-project
codegraph init
```

`codegraph init` 会一次性完成两件事：创建本地 `.codegraph/` 目录，并构建完整图谱。之后，文件监视器会自动保持图谱与代码同步，多数情况下无需再手动重建。

3.2 验证图谱状态

```
codegraph status
```

该命令报告当前索引的健康状况，包括：

- 节点（node）数量：图谱中的符号总数；
- 边（edge）数量：符号之间的关系总数；
- 文件（file）数量：已索引的源文件总数；
- 活跃的 SQLite 后端与 journal 模式（通常为 `wal`）。

若看到 `Journal:` 后面的值不是 `wal`，说明该文件系统不支持 WAL 模式，功能不受影响，可参考官方 Troubleshooting 文档确认。

3.3 首次查询

首选命令是 `codegraph explore`。它接受一段自然语言描述或一组符号名，一次返回相关符号的逐行源码，以及这些符号之间的调用路径：

```
codegraph explore "how does login work"
```

对于更精确、可脚本化的查询，有四个专门命令（均为英文符号名查询，支持 `--json` 输出）：

```
codegraph query UserService          # 按名称查找符号
codegraph callers handleRequest      # 查找谁调用了某函数
codegraph callees handleRequest      # 查找某函数调用了什么
codegraph impact AuthMiddleware      # 分析改动某符号会影响什么
```

各命令的完整参数将在第 5 章展开。

3.4 把图谱交给智能体

完成 `codegraph init` 且智能体已经通过第 2 章 `codegraph install` 接入后，不需要任何额外操作：智能体会自动发现 `.codegraph/` 目录，并开始使用 CodeGraph 的 MCP 工具回答代码问题。用户只需正常向智能体提问，例如「这个项目里用户鉴权是怎么实现的」，智能体会自动调用 `codegraph_explore` 获取答案。

3.5 全量重建与增量更新

大多数情况下文件监视器会自动同步，但在监视器被禁用、或在智能体会话之外以脚本方式修改代码后，可以使用：

```
codegraph index          # 从头全量重建
codegraph sync           # 只更新发生变化的文件
```

进阶篇

第 4 章　核心概念

本章解决的问题是：CodeGraph 内部是如何把源代码变成一张可查询图谱的，以及为什么它能够在代码改动后保持新鲜。理解本章内容是使用高级功能（第 7、8 章）的基础。

4.1 从源码到图谱的流水线

CodeGraph 的处理流程可分为三个阶段：

1. **解析（parsing）**：使用 tree-sitter 语法解析器将每个源文件解析为抽象语法树（AST），从中提取符号定义（函数、类、方法等）与符号引用。20 种语言由编译后的 Rust 内核处理，每个文件只经过一次边界跨越，速度快且结果确定。

2. **消解（resolution）**：将「引用」与「定义」匹配起来。例如，某文件中的 `userService.login()` 调用，需要被关联到另一个文件中 `UserService` 类的 `login` 方法定义。消解是跨文件调用链能够成立的关键，也是 grep 类工具做不到的部分。
3. **持久化（persistence）**：结果写入项目根目录 `.codegraph/` 下的 SQLite 数据库（`codegraph.db`），并建立 FTS5 全文检索索引。

4.2 动态派发的还原

静态解析无法确定所有调用目标。CodeGraph 通过框架感知与启发式规则补充常见的动态场景，包括回调、React 组件的重新渲染链、JSX 子组件关系等。每一条由此类规则建立的边都会标记来源（`resolvedBy: 'framework'` 或 `synthesizedBy` 等元数据），以保证图谱可追溯、不虚构。

4.3 自动同步机制

文件监视器基于操作系统原生事件工作，其时间线大致为：

```
保存 src/Widget.ts
  → 事件送达（通常 < 100 ms）
  → 2000 ms 防抖（将连续编辑合并为一次）
  → 增量同步（只处理变更文件）
```

防抖（debounce）指在固定时间窗口内将多次事件合并为一次处理。实测在一个 4400 文件的项目中，单文件变更的同步开销约为 0.3 秒；在 27000 文件的 Swift 编译器仓库中约为 0.4 秒。同步只处理变更的文件，从不重新扫描整棵目录树。

智能体侧还有两层保障：当会话打开时若索引暂时落后，会出现陈旧提示（staleness banner）；MCP 连接建立时会执行一次连接期追赶（connect-time catch-up）。这两项自 v0.9.5 起可用。

4.4 机器自适应

CodeGraph 依据运行环境的真实资源调整并行度：

- 以实际可用 CPU 核心数决定工作线程池大小（容器感知，能识别 cgroup 限制）；
- 以真实可用内存决定缓存大小；
- 以项目自身的解析与消解开销决定流水线配置。

效果是：工作站上能跑满并行流水线；而一台 2 核 6 GB 内存的云服务器也能以「保完成」的策略跑完超大规模项目（官方示例：Linux 内核约 7 万文件、200 万符号，可在 12 分钟内完成全量索引）。

4.5 数据在哪里

全部数据位于 `<项目根>/.codegraph/`，核心文件是 SQLite 数据库 `codegraph.db`。该目录属于可重建的本地索引，不应提交到版本库，建议加入 `.gitignore`。

第 5 章　CLI 查询命令详解

本章解决的问题是：在不打开智能体的情况下，如何在终端中直接、精确地查询图谱。完成本章学习后，读者能够使用全部查询类命令及其主要参数，并将 `--json` 输出用于脚本。

5.1 命令总览

CodeGraph 的查询命令分为两类：综合查询（`explore`、`node`）与精确查询（`query`、`callers`、`callees`、`impact`、`files`）。综合查询输出更完整，精确查询输出更聚焦、更适合脚本处理。

5.2 explore：一次拿到完整上下文

```
codegraph explore "how does login work"
codegraph explore UserService authenticate
```

- 参数接受一段自然语言短语，或一组空格分隔的符号名 / 文件名；
- 输出内容与 MCP 工具 `codegraph_explore` 完全一致：按文件分组的逐行源码、符号间调用路径（含动态派发跳转）、影响范围摘要。

5.3 query：按名称查找符号

```
codegraph query UserService
codegraph query UserService --kind class --limit 10
codegraph query UserService --json
```

参数	说明
`--kind`	限定符号类型，如 `class`、`function`
`--limit`	限制返回条数
`--json`	以 JSON 格式输出，便于脚本处理

5.4 callers 与 callees：调用关系双向查询

```
codegraph callers handleRequest
codegraph callees handleRequest
codegraph callers handleRequest --limit 20 --json
```

`callers` 回答「谁调用了它」，`callees` 回答「它调用了谁」。二者均接受 `--limit` 与 `--json`。

5.5 impact：影响分析

```
codegraph impact AuthMiddleware
codegraph impact AuthMiddleware --depth 3 --json
```

`--depth` 控制沿调用链向上追溯的层数。该命令用于在改动前评估影响半径，是重构与代码审查的常用前置步骤。

5.6 node：读取单个符号或文件

```
codegraph node UserService.login       # 某个符号的源码与其调用者/被调用者链
codegraph node src/auth/service.ts     # 按行号读取整个文件
```

该命令与 MCP 工具 `codegraph_node` 输出一致；对于重载（overload）同名符号，会返回所有重载的函数体。

5.7 files：查看已索引的文件结构

```
codegraph files --max-depth 2
codegraph files --pattern "auth/*" --json
```

`files` 直接从图谱返回文件树，比遍历文件系统更快，且只反映实际被索引的文件。

第 6 章　配置与定制

本章解决的问题是：当默认行为不满足需求时，如何通过一个可选配置文件调整索引范围。CodeGraph 的设计理念是零配置，因此本章内容仅在确有需要时使用。

6.1 默认行为

不编写任何配置文件时，CodeGraph 已经做了合理的默认排除：

- **依赖、构建与缓存目录**：`node_modules`、`vendor`、`dist`、`build`、`target`、`.venv`、`Pods`、`.next` 等各技术栈的常见目录，即使没有 `.gitignore` 也会被跳过。例外：若 `build` 目录中实际存放的是 TypeScript、Vue、Svelte、Go、C# 或 Rust 源码，则仍会索引（因为正常构建流程不会往这些目录写入源码）；Java/Kotlin/Scala 中名为 `build` 的包同理。
- **`.gitignore` 中的内容**：git 仓库经 git 获取；非 git 项目直接读取根目录与嵌套的 `.gitignore` 文件。
- **超过 1 MB 的文件**：通常是压缩包、打包产物或第三方大文件。

若希望把默认排除的目录纳入索引，在 `.gitignore` 中添加取反规则（如 `!vendor/`）即可。

6.2 codegraph.json：可选的项目级配置

配置文件名为 `codegraph.json`，放在项目根目录，可提交到版本库与团队共享。全部可配置项如下。

排除已被 git 跟踪的目录

`.gitignore` 只能排除未被 git 跟踪的文件；对已提交的目录（如内嵌的第三方主题、SDK），需使用 `exclude`：

```
{
  "exclude": ["static/", "**/vendor/**"]
}
```

`exclude` 的匹配规则遵循 gitignore 风格，相对项目根目录，在全量索引、增量同步与文件监视三个环节均生效，且优先级高于一切其他规则。修改后需执行 `codegraph index` 重新索引。

强制纳入 git 忽略的源码

适用场景是项目同时被 SVN、Perforce 等其他版本控制系统管理，部分源码被有意列入 `.gitignore`（对 git 而言）却仍是第一方源码。此时使用 `include`：

```
{
  "include": ["Tools/", "Local/typescript/"]
}
```

`include` 直接从磁盘发现文件，覆盖 `.gitignore`；显式 `exclude` 依然优先；内置跳过目录（`node_modules`、`dist`、`.git` 等）即使被 `include` 匹配也不会纳入。修改后需重新索引。

自定义文件扩展名

项目中若使用了非标准扩展名（如 `.dota_lua`、`.tpl`），默认不会被识别，需显式映射：

```
{
  "extensions": {
    ".dota_lua": "lua",
    ".tpl": "php"
  }
}
```

值必须是对应的支持语言 ID；映射会在内置默认之上合并，冲突时以用户配置为准（因此也可以改写内置映射，如 `".h": "cpp"`）。拼写错误或格式错误的条目只产生警告，不会中断索引。修改后需重新索引。

纳入嵌套的 git 仓库

`.gitignore` 排除的目录中若嵌套了其他 git 仓库（如克隆的参考项目），默认不会索引。若项目是一个「多仓库工作区」，希望把子仓库一起纳入同一张图谱，使用 `includeIgnored`：

```
{
  "includeIgnored": ["packages/", "services/"]
}
```

CodeGraph 会进入所列目录，并按每个子仓库自己的 `git ls-files` 索引，子仓库自身的 `.gitignore` 依然被尊重。未被 gitignore 排除的嵌套仓库会自动索引，无需配置。修改后需重新索引。

6.3 配置项速查

字段	作用	典型场景
`exclude`	将已跟踪的目录排除出索引	排除已提交的第三方主题、SDK
`include`	强制纳入被 git 忽略的文件	双 VCS 项目中被 git 忽略的第一方源码
`extensions`	为非标准扩展名指定语言	`.tpl` 按 PHP 解析
`includeIgnored`	索引被 gitignore 的目录中的嵌套仓库	多仓库工作区

精通篇

第 7 章　与 AI 智能体深度协作

本章解决的问题是：CodeGraph 的 MCP 服务器如何向智能体暴露能力，以及如何控制暴露哪些工具。完成本章学习后，读者能够理解智能体调用 CodeGraph 的机制，并按需调整工具集合。

7.1 MCP 服务器的运行方式

智能体（如 Claude Code）会自动启动 MCP 服务器进程，对应命令为：

```
codegraph serve --mcp
```

用户通常不需要手动执行该命令。其行为有一个重要约定：**当工作区中不存在 `.codegraph/` 索引时，服务器不暴露任何工具**，智能体回到常规工作方式；是否建索引始终是用户的决定。

7.2 默认唯一工具：codegraph_explore

默认情况下，MCP 服务器只暴露一个工具 `codegraph_explore`。这是有意设计：实测表明，一个瞄准准确的强工具比一排窄工具更能引导智能体直达答案，减少误选。

`codegraph_explore` 的输入是一段自然语言问题或一组符号 / 文件名，输出包含三部分：

1. 相关符号的**逐行源码**，按文件分组（与 Read 工具同形）；
2. 符号间的**调用路径**，包括 grep 无法跟踪的动态派发（回调、React 重渲染、JSX 子组件）；
3. **影响范围**（blast radius）摘要：哪些符号依赖查询目标。

对「X 是如何工作的」「X 如何到达 Y」这类问题，智能体通常一两次调用即可作答，文件读取次数为零。

7.3 其余七个工具

以下工具默认不列出（unlisted），但始终可用。它们返回的信息在 `codegraph_explore` 的响应中已内联出现，因此默认收起：

工具	用途	CLI 对应命令
`codegraph_node`	单个符号的源码与调用者/被调用者链；或按行号读取整个文件	`codegraph node`
`codegraph_search`	按名称查找符号（仅位置）	`codegraph query`
`codegraph_callers`	查找某函数的调用者	`codegraph callers`
`codegraph_callees`	查找某函数调用了什么	`codegraph callees`
`codegraph_impact`	分析改动某符号的影响	`codegraph impact`
`codegraph_files`	查看已索引的文件结构	`codegraph files`
`codegraph_status`	检查索引健康与统计	`codegraph status`

通过环境变量 `CODEGRAPH_MCP_TOOLS` 可以替换默认的工具集合，值为逗号分隔的短名列表：

```
CODEGRAPH_MCP_TOOLS=explore,node,search,callers
```

7.4 子智能体与非 MCP 环境

MCP 服务器的使用指引通过 `initialize` 响应送达主智能体，但子智能体（subagent）与不经过 MCP 的环境看不到该响应。为此，`codegraph install` 会在智能体的指令文件（`CLAUDE.md` / `AGENTS.md` / `GEMINI.md`）中写入一小段带标记围栏的说明，指向 CLI 等价命令 `codegraph explore`，使这些环境也能使用图谱。

第 8 章　进阶实战

本章解决的问题是：在四个高价值场景中使用 CodeGraph——框架路由、跨语言调用、浏览器可视化、CI 受影响测试。完成本章学习后，读者能够在真实工程中运用这些能力。

8.1 框架路由（Framework Routes）

CodeGraph 识别 17 个 Web 框架的路由定义，生成 `route` 节点，并用 `references` 边将其连接到处理函数或类。此后，查询某个控制器或视图的调用者时，会直接看到绑定它的 URL 模式。

典型识别形态举例：

框架	识别形态
Django	`urls.py` 中的 `path()` / `re_path()` / `url()` / `include()`，含 CBV `.as_view()`
FastAPI	`@app.get(...)`、`@router.post(...)` 等装饰器
Express	`app.get(...)`、`router.post(...)` 及中间件链
NestJS	`@Controller` + `@Get/@Post`，含前缀与 URI 版本控制；GraphQL `@Resolver`
Laravel	`Route::get()`、`Route::resource()`、`Controller@action`
Rails	`get '/x', to: 'users#index'` 及哈希火箭写法
Spring	`@GetMapping`、`@PostMapping`、`@RequestMapping`
Gin / chi / gorilla / mux	`r.GET(...)`、`router.HandleFunc(...)`
ASP.NET	动作方法上的 `[HttpGet("/x")]` 特性

此外，针对前端路由框架（Expo Router、Next.js、React Router、TanStack Router、Vue Router / Nuxt、SvelteKit、Angular），CodeGraph 还生成 `navigates` 边：把「发起跳转的函数」连接到「目标页面」。解析目标要求是字面量路径或可确定的路由名，无法确定的跳转不会被猜测标注。多应用仓库中，每个应用的路由只与该应用内部书写的跳转匹配。

8.2 跨语言桥接（iOS / React Native / Expo）

静态解析在语言边界处会中断。CodeGraph 内置了跨语言桥接规则，把断开的调用链重新连上：

边界	机制
Swift ↔ Objective-C	`@objc` 自动桥接规则（含初始化、属性、协议形式）与 Cocoa 介词前缀（`With` / `For` / `By` / `In` / `On` / `At`）匹配
React Native 旧桥	解析 `RCT_EXPORT_METHOD` / `RCT_REMAP_METHOD`（ObjC）与 `@ReactMethod`（Java/Kotlin）声明，建立 JS 名称到原生方法的映射
React Native TurboModules	以 `Native<X>.ts` 规格接口为基准，匹配原生实现
原生 → JS 事件	以字面量事件名为键合成事件通道（`sendEventWithName` / `.emit`）
Expo Modules	解析 Expo DSL 字面量（`Module { Name("X"); AsyncFunction("fn") }`），JS 侧 `requireNativeModule('X').fn(...)` 调用连到原生方法
Fabric / Paper 视图	JSX 组件连到 Codegen 规格与原生实现类（按 `View` / `ComponentView` / `Manager` 等后缀约定匹配）

每条桥接边都携带来源元数据（`resolvedBy` / `synthesizedBy`），保证图谱可追溯。

8.3 浏览器查看器（ui）

```
codegraph ui                    # 打开当前项目的查看器
codegraph ui ~/code/my-app      # 打开另一个已索引项目的查看器
codegraph ui --port 8080        # 指定端口（被占用则失败）
codegraph ui --no-open          # 只打印 URL，不自动打开浏览器（无头服务器、SSH）
codegraph ui --read-only        # 拒绝一切写入，包括保存的浏览轨迹
```

查看器是本地 Web 页面，监听 `127.0.0.1`：左侧为调用者，中间为符号源码，右侧为被调用者并高亮调用行。默认端口 4747，占用时自动顺延；`CODEGRAPH_BROWSER=none` 可禁止自动打开浏览器，`codegraph web` 是别名。

查看器只读取既有索引，不建索引、不修改图谱或代码；唯一的写入项是用户主动保存的浏览轨迹，存放于 `.codegraph/ui/trails/`，`--read-only` 模式下连这项也被禁止。

说明：截至 v1.6.2 核实时，`codegraph ui` 尚未随正式版发布，官方文档说明其将在后续版本提供，本节内容以届时发布的实际行为为准。

8.4 CI 中的受影响测试（affected）

```
codegraph affected src/auth/service.ts src/api/user.ts
```

`codegraph affected` 沿 import 依赖图做传递闭包，找出「本次变更的源文件会影响哪些测试文件」。将其接入 CI 可以把全量测试缩小为受影响测试。细节参数与 CI 示例以官方文档 guides/affected-tests 页面为准。

第 9 章　最佳实践

本章解决的问题是：如何在个人与团队层面把 CodeGraph 用得稳定、高效。以下均为建议，理由附后。

9.1 将 .codegraph/ 加入 .gitignore

`.codegraph/` 是完全可由源码重建的本地索引，提交到仓库只会增大仓库体积并制造合并冲突。建议在仓库根 `.gitignore` 中加入：

```
.codegraph/
```

9.2 每项目一次 init，全局一次 install

正确的分工是：`codegraph install` 只需在有新智能体接入时执行一次（全局）；`codegraph init` 在每个需要图谱的项目中执行一次。此后无需人工维护，监视器负责保持新鲜。

9.3 大型第三方目录：排除而非放任

已提交到仓库的第三方目录（主题、SDK、生成代码）会显著膨胀图谱并稀释查询精度。使用 `codegraph.json` 的 `exclude`（第 6 章）将其排除，重新索引即可。

9.4 团队共享配置

`codegraph.json`、扩展名映射应提交到版本库，保证团队成员（以及各成员的智能体）面对相同的索引范围。代码之外，`.gitignore` 规则与 `codegraph.json` 共同构成团队的「索引契约」。

9.5 资源受限环境的预期

CodeGraph 会按真实可用资源自我调整。在 2 核 / 6 GB 的环境中，大仓库首次全量索引可能耗时较长（官方基准：Linux 内核规模约 12 分钟），属预期行为；索引完成后，日常增量同步开销极小。

9.6 长会话的上下文占用

官方测量指出：CodeGraph 一次返回的是密集的逐行源码载荷，回答完后仍驻留在上下文窗口中；多轮长会话结束时，残留检索上下文约占比文件阅读方式多 80%。若在小窗口模型上进行超长会话，建议控制会话长度或使用更大的上下文窗口。

第 10 章　排错、升级与卸载

本章解决的问题是：常见故障的自查路径，以及如何干净地升级或移除 CodeGraph。

10.1 常见问题与处理

现象	可能原因与处理
智能体看不到 CodeGraph 工具	项目中没有 `.codegraph/` 索引（服务器在无索引时有意不暴露工具），或智能体未重启。执行 `codegraph init` 并重启智能体
索引疑似陈旧	查看智能体中的陈旧提示；必要时执行 `codegraph sync`。会话重连时也会自动追赶
修改 codegraph.json 后未生效	这类配置只在全量索引时读取。执行 `codegraph index` 重新索引
索引被锁、无法更新	执行 `codegraph unlock` 移除残留的锁文件
`status` 显示 journal 不是 `wal`	文件系统不支持 WAL，功能不受影响，可查阅官方 Troubleshooting
查询结果为空	确认目标语言的扩展名被支持；非标准扩展名需在 `codegraph.json` 的 `extensions` 中映射

10.2 升级

```
codegraph upgrade            # 升级到最新版
codegraph upgrade --check    # 只检查，不升级
codegraph version            # 确认当前版本
```

10.3 卸载

从所有已配置的智能体中移除 CodeGraph（MCP 配置、指令文件中的说明、权限项），并同时移除 CLI 本体；移除前会列出将要删除的内容：

```
codegraph uninstall
```

常用变体：

```
codegraph uninstall --keep-cli       # 只移除智能体配置，保留 CLI
codegraph uninstall --target=cursor  # 只从指定智能体移除
codegraph uninstall --yes            # 非交互执行
```

若还想删除某个项目的图谱索引：

```
codegraph uninit
codegraph uninit --force     # 跳过确认提示
```

`uninstall` 不会触碰项目内的 `.codegraph/` 目录；`uninit` 则按项目删除。

附录

附录 A　CLI 命令速查表

命令	作用	常用参数
`codegraph`（无子命令）	运行交互式安装器	—
`codegraph install`	接入 AI 智能体	`--yes`、`--target=`、`--location=`、`--no-permissions`、`--print-config <id>`
`codegraph uninstall`	反向移除智能体配置与 CLI	`--keep-cli`、`--target=`、`--yes`
`codegraph init [path]`	初始化项目并建图谱	—
`codegraph uninit [path]`	移除项目图谱	`--force`
`codegraph index [path]`	全量重建索引	`--force`、`--quiet`、`--verbose`
`codegraph sync [path]`	增量更新	`--quiet`
`codegraph status [path]`	查看统计与健康状态	`--json`
`codegraph unlock [path]`	移除阻塞索引的残留锁文件	—
`codegraph query <search>`	按名称查符号	`--kind`、`--limit`、`--json`
`codegraph explore <query>`	自然语言/符号名综合查询	—
`codegraph node <symbol|file>`	读单符号源码或按行读文件	—
`codegraph files [path]`	查看已索引文件结构	`--format`、`--filter`、`--pattern`、`--max-depth`、`--json`
`codegraph callers <symbol>`	查谁调用了它	`--limit`、`--json`
`codegraph callees <symbol>`	查它调用了谁	`--limit`、`--json`
`codegraph impact <symbol>`	影响分析	`--depth`、`--json`
`codegraph affected [files...]`	查变更影响的测试文件	见官方 guides
`codegraph ui [path]`	打开浏览器查看器（未随正式版发布）	`--port`、`--no-open`、`--read-only`
`codegraph daemon`	管理后台守护进程（别名 `daemons`）	—
`codegraph telemetry [on|off]`	匿名遥测开关	亦可用 `CODEGRAPH_TELEMETRY=0` 或 `DO_NOT_TRACK=1`
`codegraph upgrade [version]`	升级	`--check`、`--force`
`codegraph version`	显示版本（`-v`、`--version` 等价）	—
`codegraph serve --mcp`	启动 MCP 服务器（由智能体自动拉起）	—
`codegraph help [command]`	帮助	—

附录 B　术语表

术语	定义
符号（symbol）	代码中可命名的实体：函数、类、方法、变量、路由等，是图谱的节点
边（edge）	符号间的关系：调用（calls）、引用（references）、跳转（navigates）等
知识图谱（knowledge graph）	由符号节点与关系边构成的图，CodeGraph 对代码库的结构化表示
索引（index）	图谱数据的持久化结果，位于项目根目录 `.codegraph/` 下的 SQLite 数据库
MCP（Model Context Protocol）	让 AI 智能体以标准协议调用外部工具服务器的开放协议
消解（resolution）	将符号引用与符号定义跨文件匹配的过程，决定调用链是否完整
影响半径（blast radius）	依赖某符号的全部符号集合，用于评估改动的波及范围
防抖（debounce）	将时间窗口内的多次文件变更事件合并为一次同步处理的机制
桥接（bridging）	通过框架规则将跨语言（如 Swift ↔ ObjC、JS ↔ 原生模块）的调用连接起来的机制
全量索引 / 增量同步	`index` 从零重建全部图谱；`sync` 只处理变更文件。自动监视器开启时通常无需手动执行二者